FGV EMAp

João Pedro Jerônimo

Aprendizado Profundo

Revisão para A2

Rio de Janeiro

2026

Conteúdo

1 Autoencoders 3
1.1 Introdução 4
1.2 Autoencoders 4
1.2.1 PCA (Principal Component Analysis) 5
1.2.2 AE (Autoencoder) 5
1.3 Autoencoders Convolucionais 5
1.4 Denoising Autoencoders 6
1.4.1 Speckel suppression in SAR Images 7
1.5 Variational Autoencoders 7
1.5.1 Treinamento 9
1.5.2 Geradores de Clima 11
1.5.3 Realismo 12
2 Aprendizado de Similaridade 13
2.1 Limitações de Classificadores 14
2.2 One-Shot Learning 14
2.3 Similarity Learning 15
2.4 Redes Siamesas 17
2.4.1 Constrastive Loss 17
2.4.2 Weighted Double Margin Contrastive (WDMC) Loss 18
2.4.3 Loss Tripla 18
2.5 Improving Similarity Learning 20
2.5.1 Melhorar a Amostragem 20
2.5.2 Melhorar a Loss 22
2.5.3 Ensemble Methods (Divide and Conquer) 23
3 Aprendendo em Dados Não-Rotulados 25

Autoencoders

1.1 Introdução

No mundo real, os dados que recebemos são frequentemente incompletos, ruidosos ou de alta dimensionalidade. Então existe a necessidade de construir conhecimento a partir de dados imperfeitos. Um dos principais exemplos é a **falta de dados rotulados**. A maioria dos algoritmos de machine learning básicos necessitam que, pelo menos uma parte dos dados, possua rótulos e, a partir do padrão apresentado entre os dados rotulados, o modelo consegue **aprender**.

No âmbito específico de **modelos generativos**, como já comentamos na aula de GANs, partimos do pressuposto de uma **distribuição de probabilidade**, que gera os dados da vida real. No entanto, essa distribuição é desconhecida, complexa e de alta dimensionalidade. Para contornar esse problema, assumimos uma distribuição mais simples, chamada **distribuição latente**, que é mapeada para a distribuição de probabilidade dos dados reais com o intuito de aproximá-la. Então os modelos generativos tentam responder à pergunta:

Como podemos aprender uma p_{model} *similar à* p_{dados}

Uma grande vantagem dessa abordagem é que, como temos o conceito de distribuição, podemos ensinar o modelo **a partir dos outliers**. Mas por que isso seria interessante? A lógica vem de que, como outliers são casos muito raros e de difícil identificação, se o modelo consegue entender bem essas situações, os casos mais comuns são identificados com extrema facilidade. Em um conjunto de imagens de trânsito, a maioria das imagens seriam em climas ensolarados, um trânsito calmo e tranquilo, no entanto, alguns edge cases como esses:



Figura 1: Edge case de imagens de trânsito

1.2 Autoencoders

Um Autoencoder é uma rede neural feed-forward projetada para redução da dimensionalidade dos dados e, a partir das features aprendidas na dimensionalidade reduzida, reconstruir os dados originais.

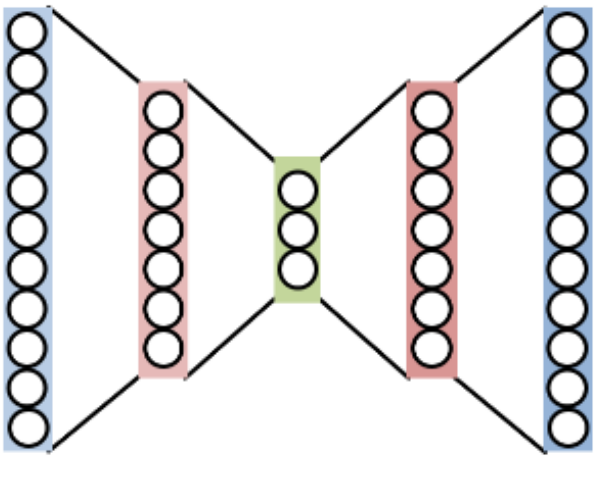


Figura 2: Desenho de esqueleto de Autoencoder

Antes de entrarmos nos conceitos mais modernos de Autoencoders, vamos ver alguns autoencoders que eram utilizados antes

1.2.1 PCA (Principal Component Analysis)

Não vamos entrar nos detalhes matemáticos do PCA, estes já foram abordados no *Resumo de Machine Learning A*. Mas intuitivamente, ele reduz a dimensionalidade dos dados através dos seus valores singulares, pegando as features **mais informativas**. Ou seja, ele faz uma **projeção linear** dos dados.

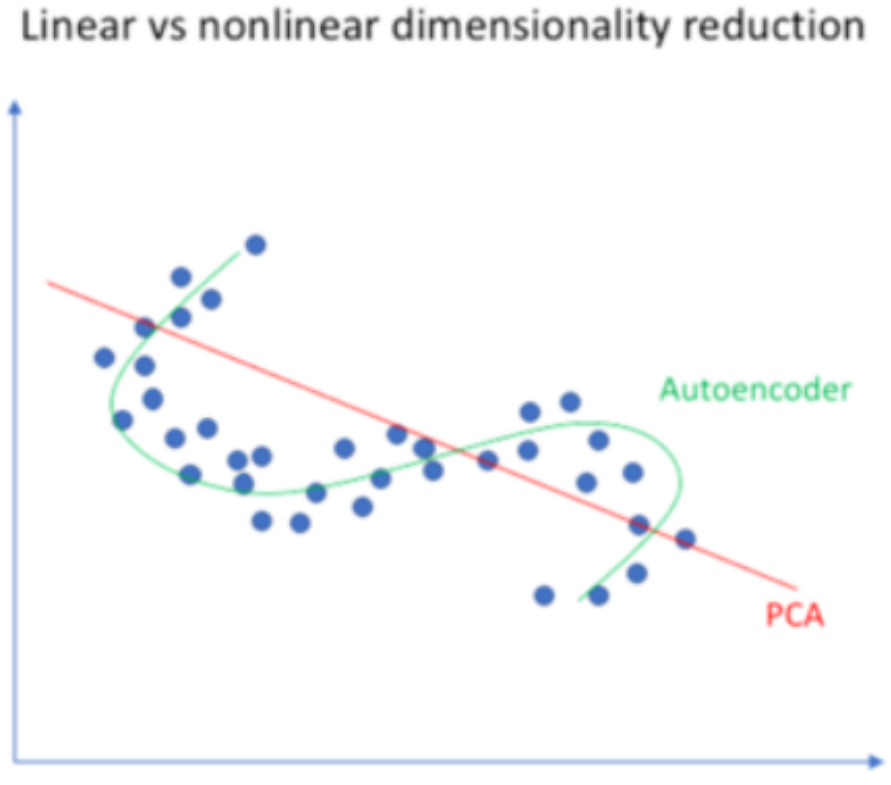


Figura 3: Exemplo de PCA em 2D para redução de dimensionalidade para 1D

1.2.2 AE (Autoencoder)

Já os autoencoders, tentam pegar as não-linearidades dos dados a partir das funções não-lineares das suas camadas, de forma que a loss que eles tentam otimizar foca em diminuir o quão distante os dados originais são dos gerados.

$$\mathcal{L}(W) = \frac{1}{N}\sum_{i}\|x_i - x_i'\|^2 + \lambda R(W) \tag{1}$$

A real é que não estamos interessados necessariamente nos dados gerados no output, mas sim no **bottleneck**, que é a camada de menor dimensionalidade no interior do autoencoder. Essa camada é a que contém as features mais informativas dos dados, e é a partir dela que podemos fazer **downstream tasks** como classificação, regressão, etc.

1.3 Autoencoders Convolucionais

Esses possuem um foco em imagens, onde as features são reduzidas através de camdas convolucionais. Onde são reduzidas e geradas novamente para se aproximar da imagem original.

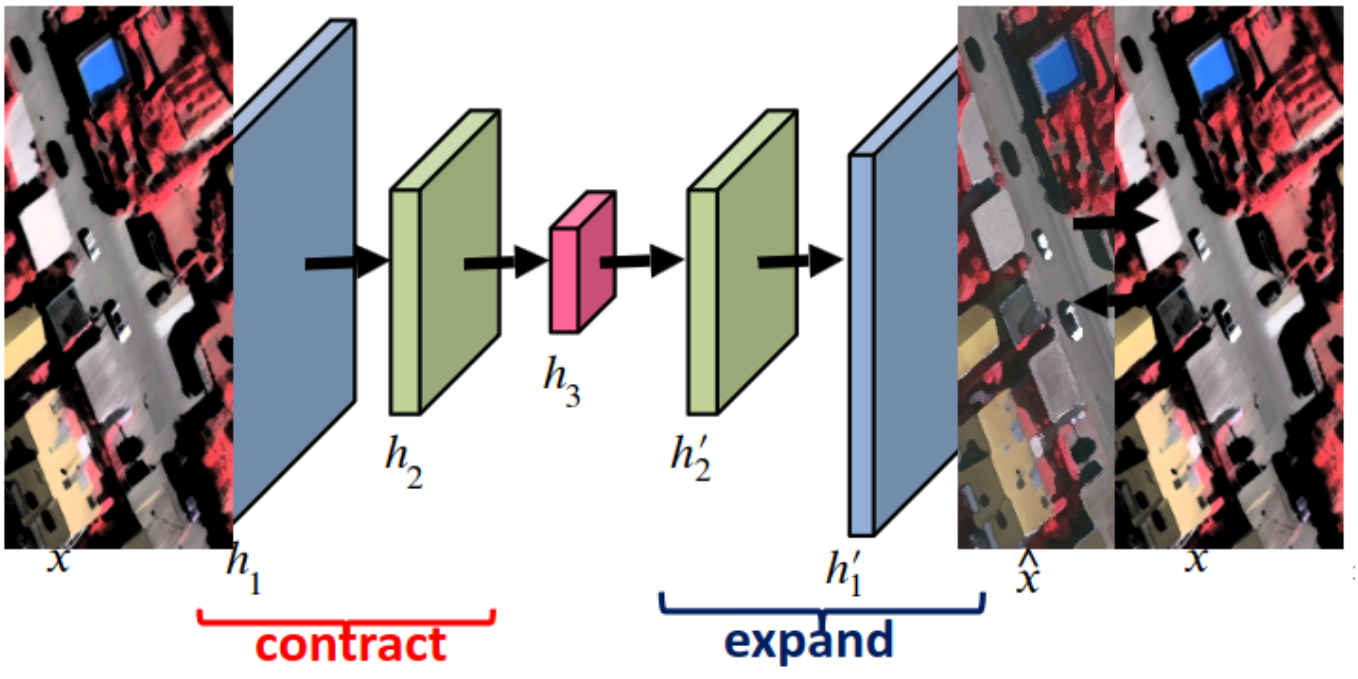


Figura 4: Exemplo de Autoencoder Convolucional

1.4 Denoising Autoencoders

Os denoising autoencoders são uma variação de autoencoders onde as imagens inseridas no input são **corrompidas** com algum tipo de ruído. A ideia por trás disso vem de que, se o modelo consegue aprender a reconstruir imagens ruidosas, imagens mais limpas são reconstruídas com mais facilidade. Não só isso, mas o encoder fica robusto à ruídos, o que é interessante para **downstream tasks** como classificação, onde imagens ruidosas podem ser classificadas com mais facilidade.

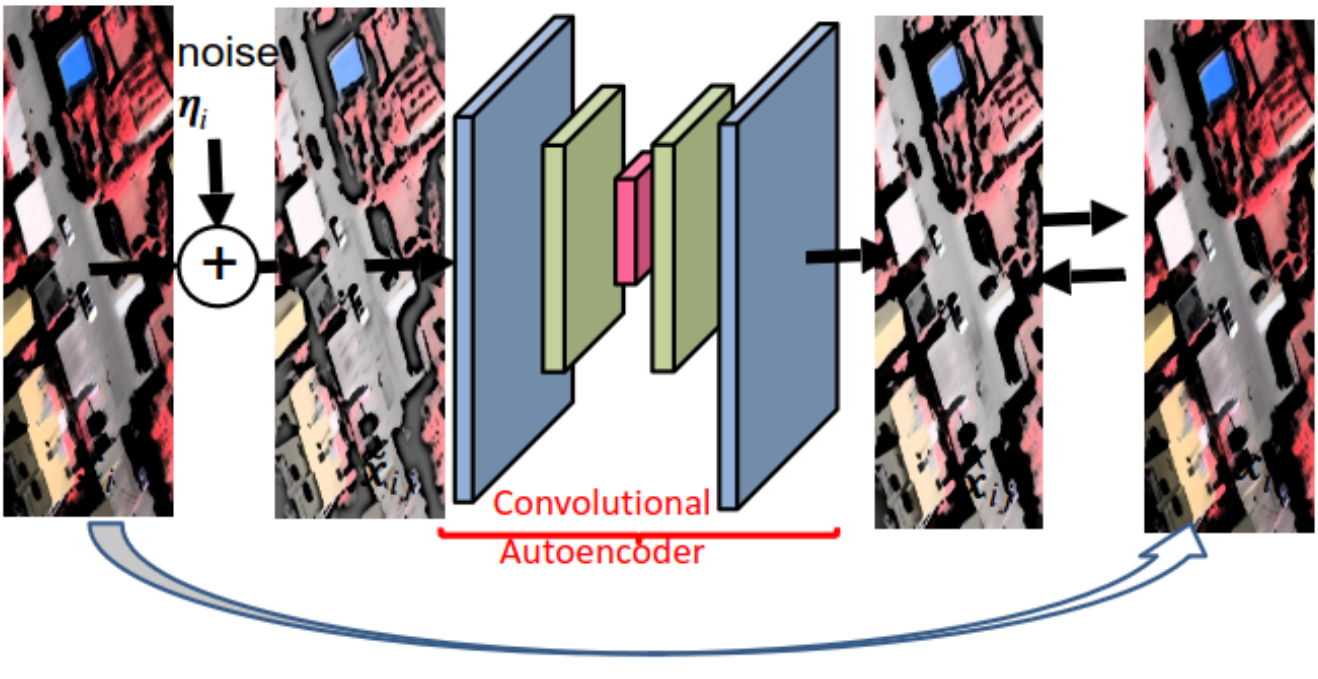


Figura 5: Exemplo de Denoising Autoencoder

Aqui, a loss é utilizada igual anteriormente, o ruído é aplicado apenas na entrada, assim o modelo não entende que o ruído é parte da imagem, mas sim que ele deve ser removido.

1.4.1 Speckel suppression in SAR Images

Os satélites de radar (SAR — **Synthetic Aperture Radar**) enviam pulsos de micro-ondas para a Terra e medem o sinal refletido. Devido ao fenômeno de interferência de ondas ao baterem no solo, a imagem gerada vem naturalmente com um ruído granulado chamado **speckle** (parece um "sal e pimenta" cobrindo a imagem).

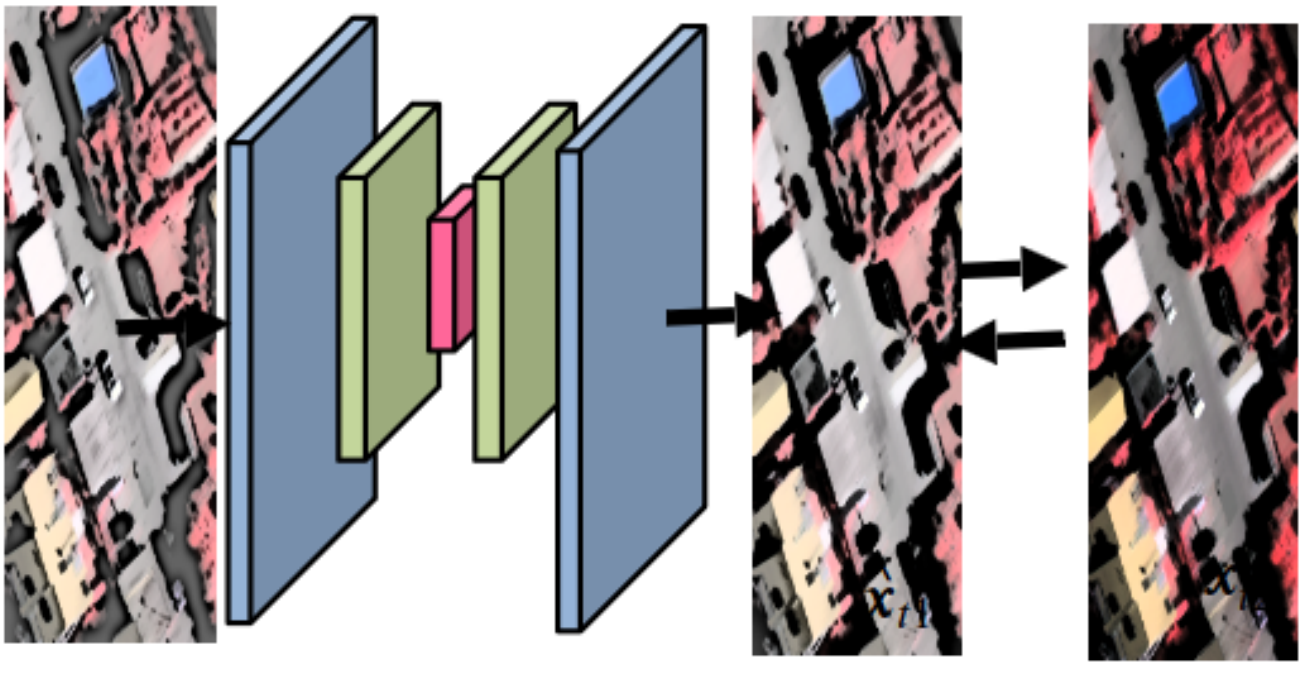


Figura 6: Exemplo de supressão de speckle em imagens SAR

No entanto, como falamos antes, o denoising autoencoder **ainda necessita de imagens limpas** para a sua loss, no entanto, com imagens de satélite, não temos imagens limpas. O que fazer então? Nós partimos de um pressuposto, se uma imagem foi tirada em um dia específico do ano, a imagem do mesmo local 1 dia depois (ou um período de tempo fixo determinado pelos cientistas) é **quase** a mesma imagem. Com essa premissa, assumimos que o ruído gerado em ambas as imagens é **aleatório** e **independente**, de tal forma que fazemos nossa rede receber a primeira imagem e tentar gerar a segunda. Isso remove o ruído justamente pela independência, fazendo com que a rede **não consiga aprender o padrão do ruído**, mas sim o padrão da imagem.

1.5 Variational Autoencoders

Se relembrarmos do que vimos em GANs, com os geradores, conseguimos gerar imagens a partir de uma distribuição latente.

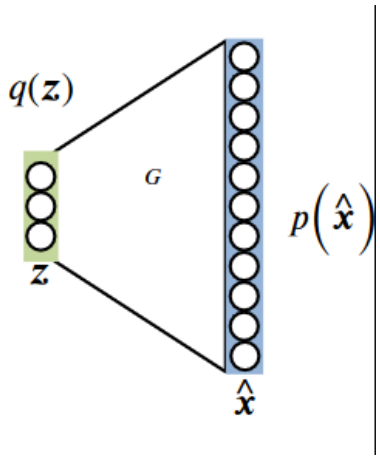


Figura 7: Exemplo de esqueleto de GAN

Nós temos a função $G : Z \rightarrow X$ que gera as minhas imagens. No entanto, não possuímos $G^{-1} : X \rightarrow Z$. E porque essa função poderia ser interessante? Se tivéssemos essa função, poderíamos modificar z para obter variações em características específicas da imagem/objeto desejado sujeitas à nossa vontade, por exemplo, mudar especificamente a cor dos olhos de uma pessoa.



Figura 8: Exemplo de variações de imagens geradas a partir de uma distribuição latente

Vamos assumir por um momento que a distribuição das imagens x advém de uma distribuição de uma variável latente z que **não conseguimos observar**. Em um esquema rápido, ficaria estruturado da seguinte forma:

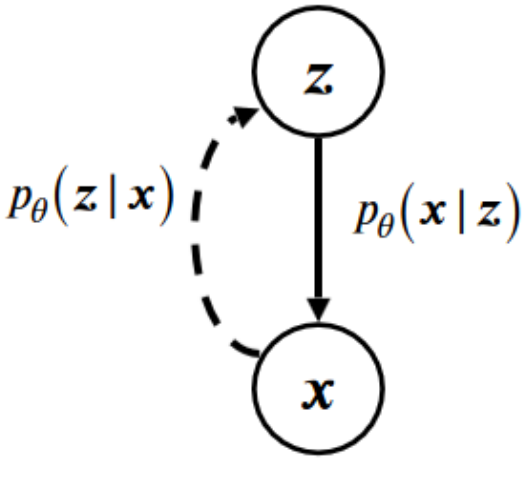


Figura 9: Exemplo de esqueleto de VAE

Nessa estrutura, algums métodos permitem estimar $p_\theta(x|z)$, no entanto, estimar $p_\theta(z|x)$ é **intratável**. A alternativa para isso, é utilizarmos uma distribuição **tratável** e **conhecida** $q_\varphi(z|x)$ e tentarmos, de alguma forma, aproximar $p_\theta(z|x)$ com $q_\varphi(z|x)$.

Na arquitetura dos Variational Autoencoders, em vez de tentar determinar features do espaço latente, tentamos associar cada entrada da distribuição com os parâmetros da distribuição conhecida $q_\varphi(z|x)$. De forma que depois do ajuste de φ, geramos novos valores para a variável latente z e, a partir da região do decoder (que representa $p_\theta(x|z)$), conseguimos gerar novas imagens.

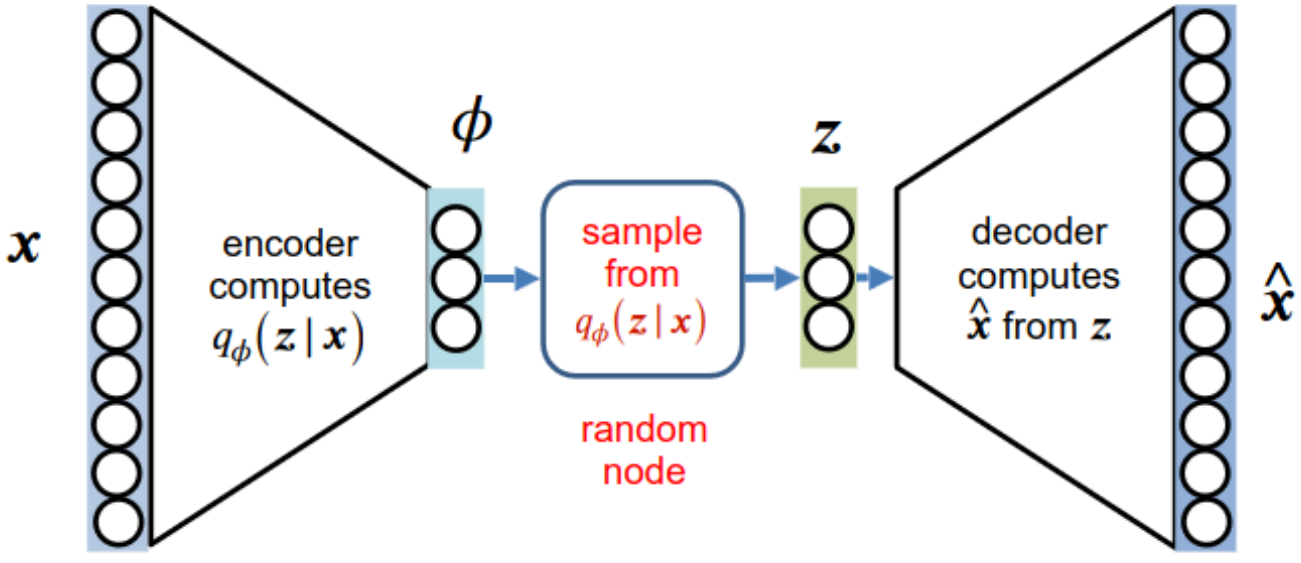


Figura 10: Exemplo de esqueleto de VAE

A loss dessa estrutura comprime dois termos principais

$$\mathcal{L}(\theta, \varphi, x) = \mathcal{L}_{\text{rec}}(\theta, \varphi, x) + \mathcal{L}_{\text{reg}}(\varphi) \tag{2}$$

o primeiro termo mede o quão bem o modelo consegue reconstruir a entrada, podemos usar, por exemplo:

$$\mathcal{L}_{\text{rec}}(\theta, \varphi, x) = \sum_i \|x_i - x_i'\|^2 \tag{3}$$

enquanto o termo de regularização mede o quão próximo a distribuição $q_\varphi(z|x)$ está da distribuição $p(z)$, que é a distribuição latente que assumimos. Para isso, podemos utilizar a divergência de Kullback-Leibler:

$$\mathcal{L}_{\text{reg}}(\varphi) = D_{\text{KL}}\big(q_\varphi(z|x) \parallel p(z)\big) \tag{4}$$

onde normalmente definimos $p(z)$ como uma distribuição normal multivariada padrão, ou seja, $p(z) = N(0, I)$. Essa regularização é importante pois o espaço latente deve ter duas características principais:

1. Continuidade: pequenas mudanças em z devem resultar em pequenas mudanças em x.
2. Completude: qualquer ponto no espaço latente deve gerar uma amostra válida e com significado.

$$D_{\text{KL}}\big(q_\varphi(z|x) \parallel p(z)\big) = \frac{1}{2}\sum_{j=1}^{J}\big(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2\big) \tag{5}$$

Utilizar a distribuição normal encoraja o encoder a distribuir os pontos no centro do espaço latente e penaliza pontos que se formam em clusters longe do centro.

1.5.1 Treinamento

Agora o ponto principal, como fazemos para treinar um VAE? De início parece fácil. Temos nossa loss, basta fazer um backpropagation pelas suas camadas. No entanto, quando chegamos na camada de amostragem, temos um problema.

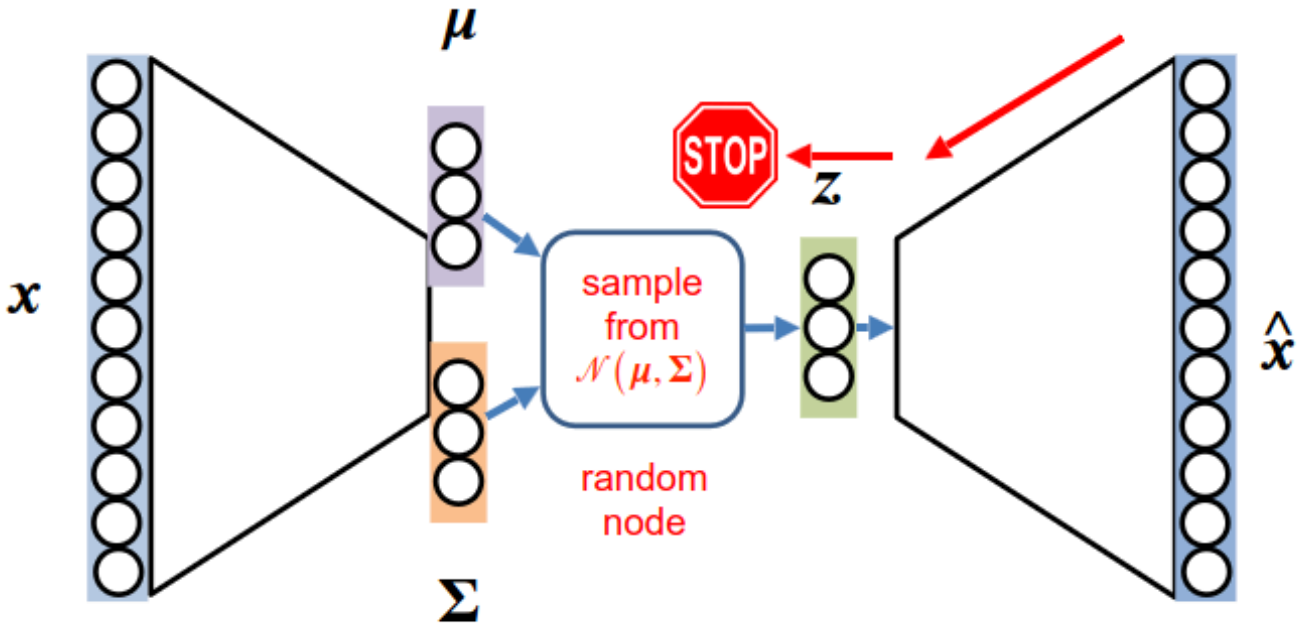


Figura 11: Ilustração do problema de backpropagation em VAE

O problema é que, como dito antes, nós precisamos amostrar da distribuição $q_\varphi(z|x)$, mas a amostragem não é diferenciável, então não conseguimos fazer backpropagation. A solução para isso é o **reparametrization trick**, onde ao invés de amostrar diretamente de $q_\varphi(z|x)$, nós amostramos de uma distribuição normal padrão e depois aplicamos uma transformação linear para obtermos a amostra desejada.

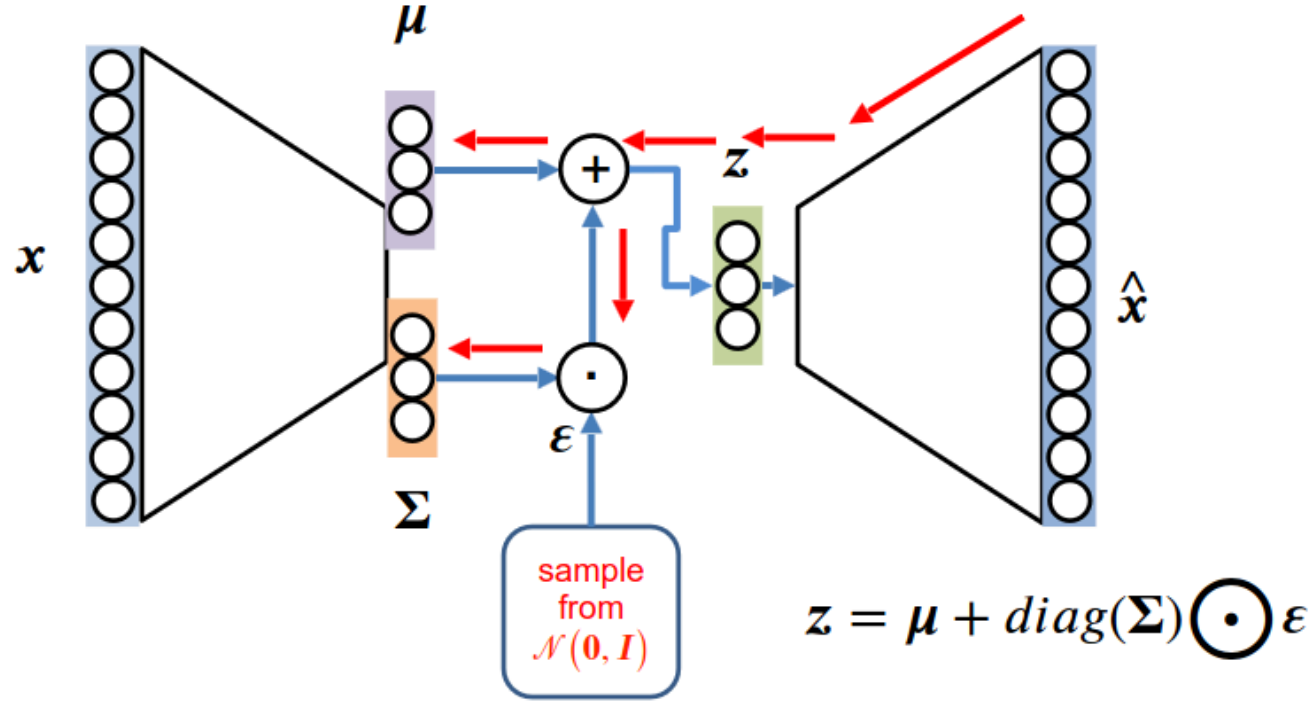


Figura 12: Exemplo de reparametrization trick

Acontece que, se amostrarmos $\varepsilon \sim N(0, 1)$, a variável latente z pode ser reparametrizada como:

$$z = \mu + \text{diag}(\Sigma) \cdot \varepsilon \tag{6}$$

de forma que isso é igual a amostrar z diretamente de uma $N(\mu, \Sigma)$ com a vantagem de podermos diferenciar com relação aos parâmetros.

1.5.2 Geradores de Clima

Uma aplicação interessante de VAE é a geração de imagens de clima. Queremos gerar **Weather Fields**, que é uma representação multidimensional (espaço + tempo) de variáveis meteorológicas, como vento, temperatura, pressão, etc. sobre uma região.

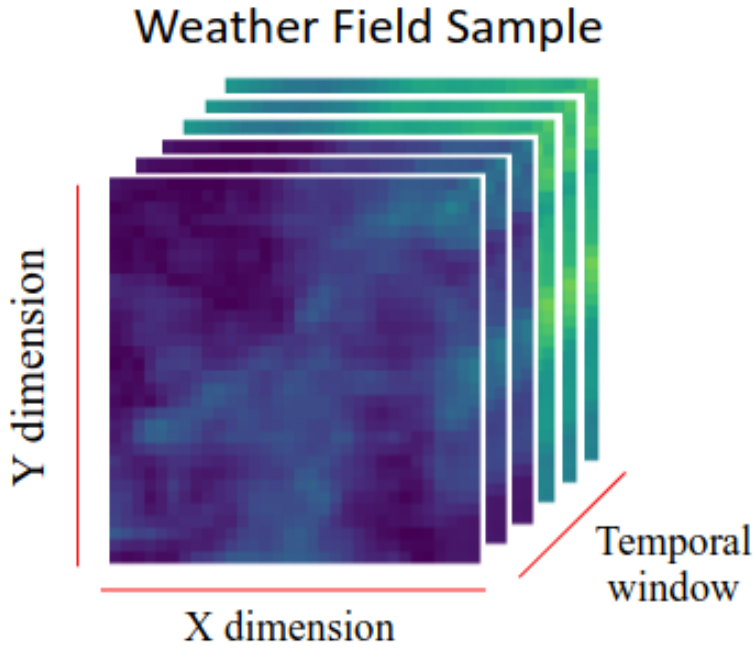


Figura 13: Exemplo de Weather Fields

Meu objetivo é, de alguma forma, ter um controle sobre o tipo de clima que eu gostaria de gerar em uma região. É aí que entra a sacada. Com os VAE, já que eu forço a distribuição latente a ser uma distribuição normal multivariada, o VAE consegue concentrar os climas comuns no centro do espaço latente e os climas extremos (e raros) nas caudas da distribuição.

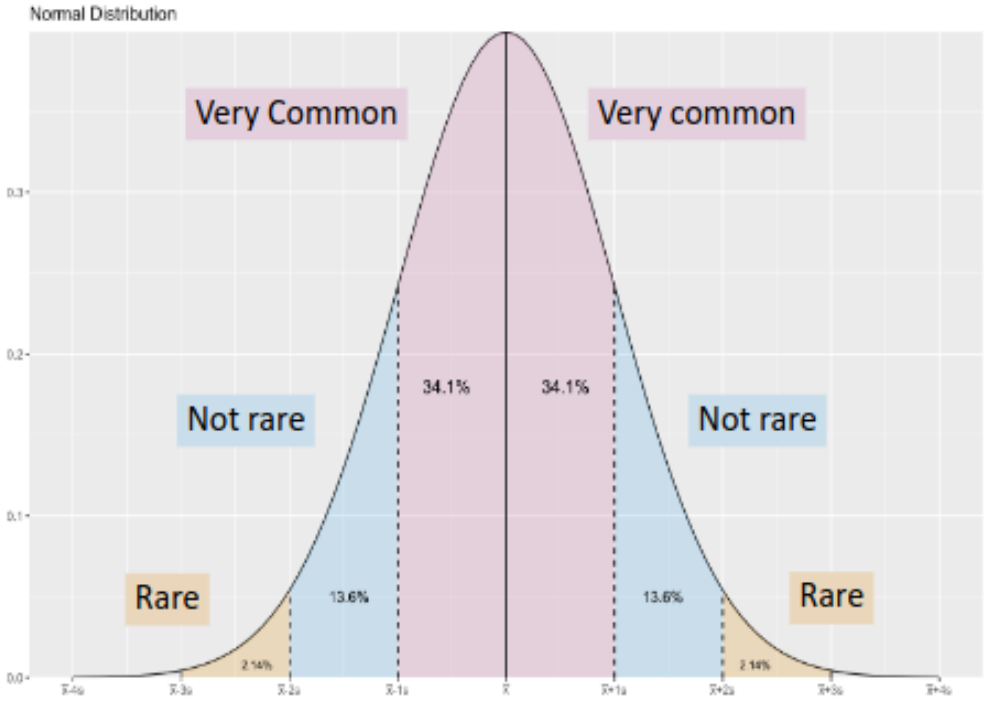


Figura 14: Exemplo de distribuição de climas no espaço latente

Dessa forma, se eu quiser simular eventos comuns naquela região, eu posso amostrar do centro da distribuição, enquanto se eu quiser simular eventos extremos, eu posso amostrar das caudas da distribuição. Isso é bom também pois não precisamos definir exatamente o que significa ser raro, o próprio VAE aprende isso a partir da distribuição latente.

Utilizando de gráficos QQ, os cientistas conseguiram mostrar que as imagens geradas pelo VAE dos eventos extremos se aproximavam muito com os dados reais observados.

1.5.3 Realismo

Para melhorar o realismo das imagens, conseguimos fazer uma mistura de conceitos que já vimos, usando GANs em conjunto com VAE. A ideia é que o GERADOR do GAN é o DECODER do VAE e o DISCRIMINADOR do GAN é um classificador que tenta diferenciar imagens reais de imagens geradas pelo DECODER. Dessa forma, o DECODER é forçado a gerar imagens mais realistas para enganar o DISCRIMINADOR.

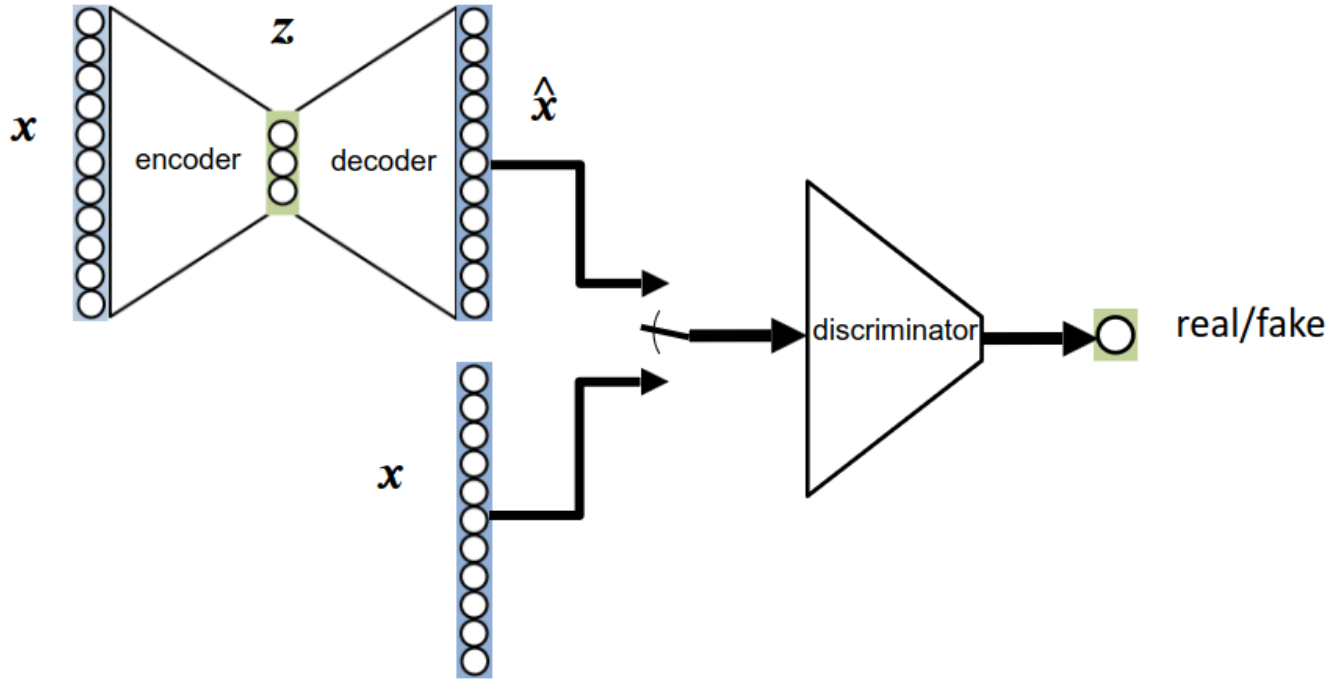


Figura 15: Exemplo de VAE-GAN

Aprendizado de Similaridade

2.1 Limitações de Classificadores

Tipicamente, quando a classificação envolve fitar o modelo em um conjunto de dados. No entanto, o modelo costuma pegar os padrões estruturais presentes naqueles dados. Por exemplo, se ele identifica imagens de animais, ele pode aprender que animais possuem olhos, patas, cauda, etc. No entanto, se ele for apresentado uma imagem do mesmo animal em posições diferentes, ele pode ter dificuldades de classificar corretamente.



Figura 16: Exemplo de limitação de classificadores

Outra limitação que podemos citar é o retreino desnecessário. Se temos um banco de dados e queremos adicionar novas imagens ou uma nova classe para enriquecer meu modelo, será necessário **treiná-lo do zero** novamente.

2.2 One-Shot Learning

Esse ramo de classificação tenta resolver o problema anterior. Nele, você tenta comparar duas imagens nunca vistas antes e determinar se elas são da mesma classe ou não. Ou seja, você não precisa fitar o modelo em um conjunto de dados, mas sim aprender a **comparar imagens**. Uma aplicação bem comum desse tipo de aprendizado é em Biometrias, como reconhecimento facial, onde você quer comparar a imagem de uma pessoa com a imagem que você tem no banco de dados para determinar se é a mesma pessoa ou não.

Uma forma mais simplificada utilizada em tarefas de visão computacional é tentar criar bijeções entre as imagens a partir de pontos específicos. A imagem a seguir exemplifica esse processo

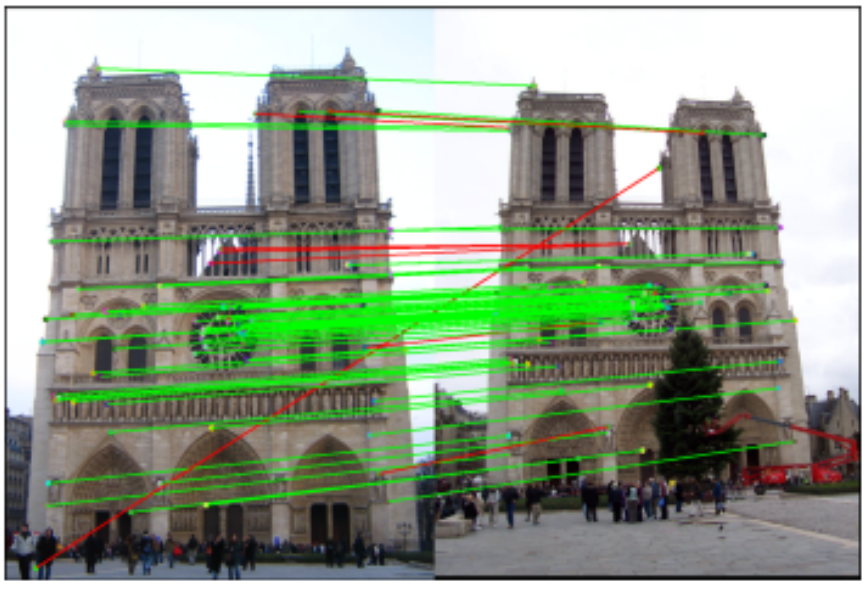


Figura 17: Exemplo de One-Shot Learning

Em diante, vamos utilizar o problema de **reconhecimento facial** como exemplo para explicar os conceitos do resto desse capítulo

2.3 Similarity Learning

Para contornar o problema da verificação, podemos trocar a estrutura do classificador! Podemos criar uma DNN que recebe duas imagens e retorna um valor entre 0 e 1, onde 0 significa que as imagens são diferentes e 1 significa que as imagens são iguais e utilizamos a loss da **Binary Cross Entropy** para treinar o modelo.

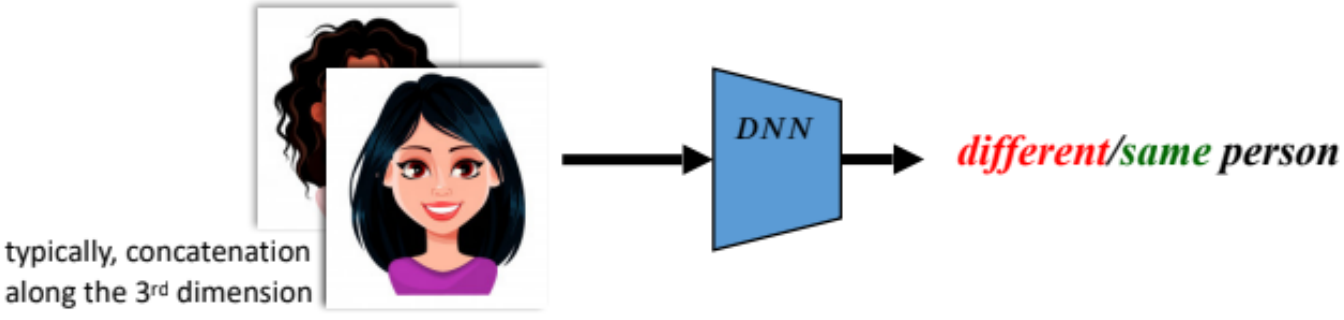


Figura 18: Exemplo de Similarity Learning

Mas essa abordagem não faz com que a rede aprenda a **medir o quão parecidas as imagens são**. Para esse objetivo, entramos em dois pontos que esses modelos de similaridade devem atingir:

1. **Aprender Métrica de Distância**: Dependendo do problema, precisamos de uma métrica de distância diferente. Por exemplo, para reconhecimento facial, a distância euclidiana é suficiente, mas para reconhecimento de imagens médicas, a distância de Mahalanobis pode ser mais interessante. Esse é um tópico de pesquisa ativo, onde o objetivo é aprender a métrica de distância mais adequada para o problema. O ponto é que precisamos de uma função que assinale pontuações altas/baixas para imagens parecidas/diferentes.
2. **Aprender Representações**: A rede precisa aprender a extrair features das imagens que sejam importantes e se encaixem nas características da métrica abordada. Isso é, atualmente, razoavelmente fácil de se fazer com redes profundas, pois elas próprias conseguem aprender as features importantes de acordo com a tarefa.

Podemos resumir o aprendizado de similaridade em uma frase: **Aprender um espaço de embeddings $\mathbb{R}^k$ onde imagens de pessoas parecidas estão próximas e imagens de pessoas diferentes estão distantes**.

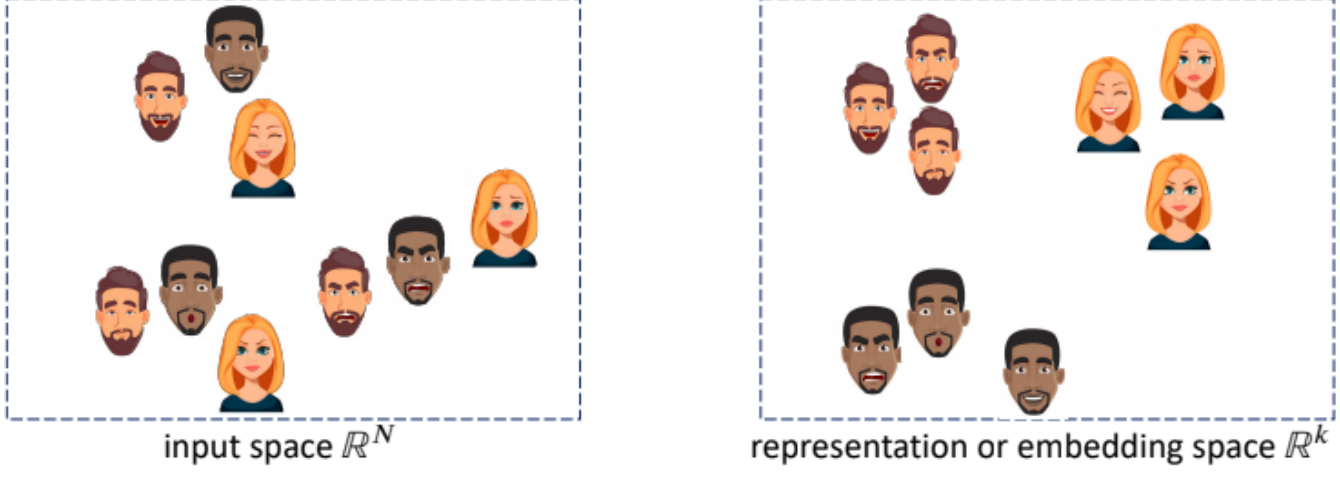


Figura 19: Exemplo de espaço de embeddings

Com isso em mente, podemos separar o processo de aprendizado em alguns passos

1. Precisamos encontrar um mapeamento $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ que mapeia as imagens de entrada do $\mathbb{R}^n$ para o espaço de embeddings $\mathbb{R}^k$. Precisamos aprender um mapeamento que satisfaça a característica de agrupamento citada anteriormente.

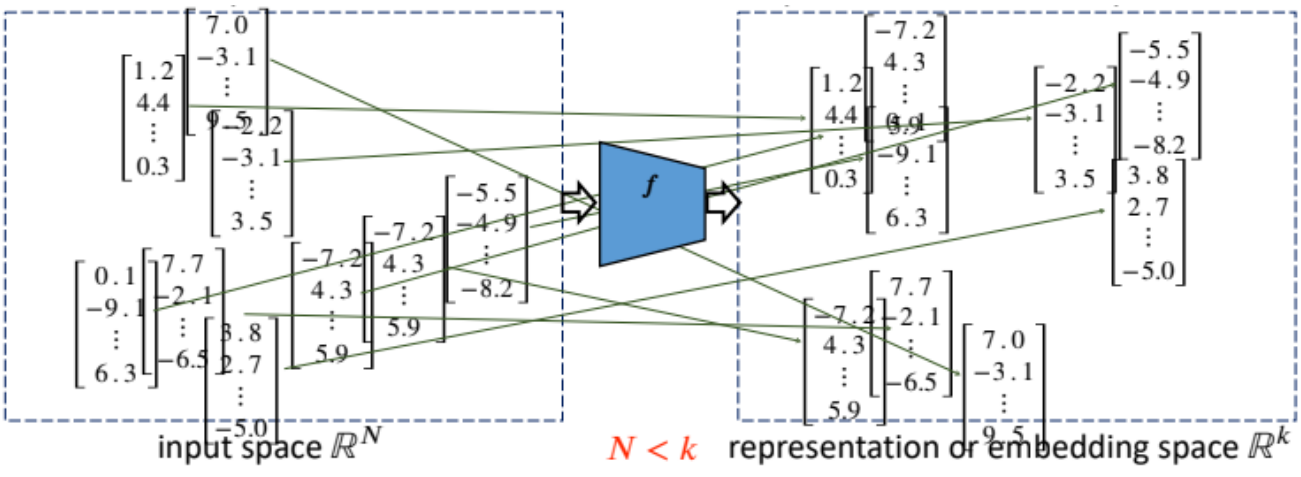


Figura 20: Exemplo de mapeamento para o espaço de embeddings

2. Agora é necessário escolher uma função de distância $d : \mathbb{R}^k x \mathbb{R}^k \rightarrow \mathbb{R}$ que mede a distância entre os embeddings. Essa função precisa ser escolhida de acordo com o problema, como citado anteriormente.

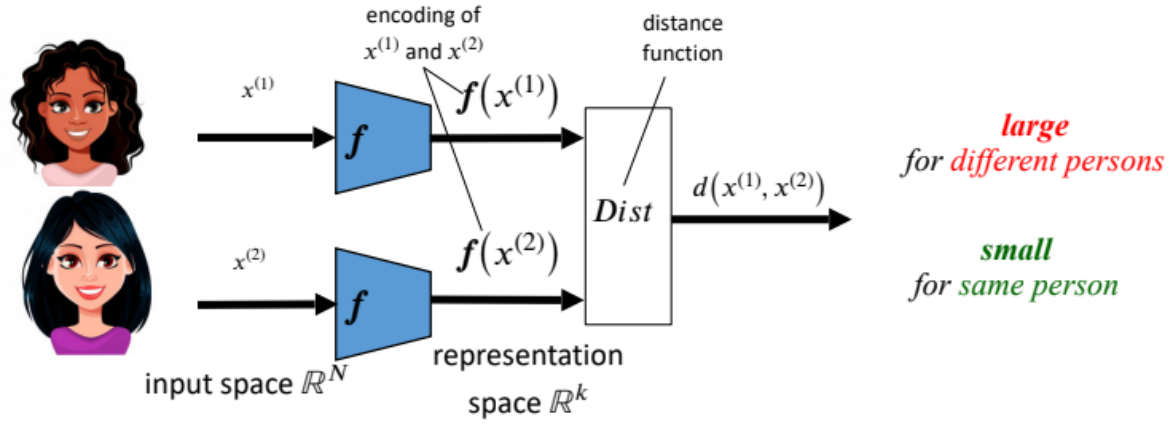


Figura 21: Exemplo de função de distância no espaço de embeddings

3. Agora precisamos definir um **threshold** τ que determina se duas imagens são da mesma classe ou não. Ou seja, se a distância entre os embeddings for menor que o threshold, as imagens são da mesma classe, caso contrário, são de classes diferentes.

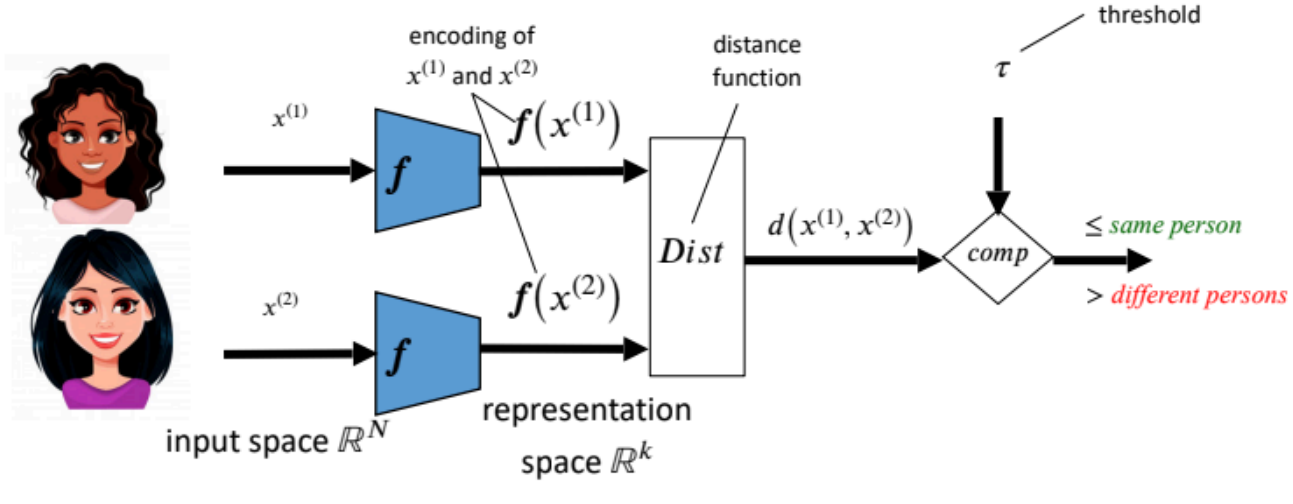


Figura 22: Exemplo de threshold no espaço de embeddings

2.4 Redes Siamesas

Redes siamesas é uma arquitetura de rede neural composta de duas redes **idênticas** que compartilham os mesmos pesos e parâmetros. O objetivo é que ambas as redes recebam uma imagem/entrada cada e realizem o mapeamento de $\mathbb{R}^n \rightarrow \mathbb{R}^k$. Depois que ambas fazem o mapeamento, tentamos medir o quão similares são os embeddings gerados. A ideia é que, se as imagens forem da mesma classe, os embeddings devem estar próximos, caso contrário, devem estar distantes.

$$d\left(x^{(1)}, x^{(2)}\right) = \left\| f\left(x^{(1)}\right) - f\left(x^{(2)}\right) \right\|^2$$

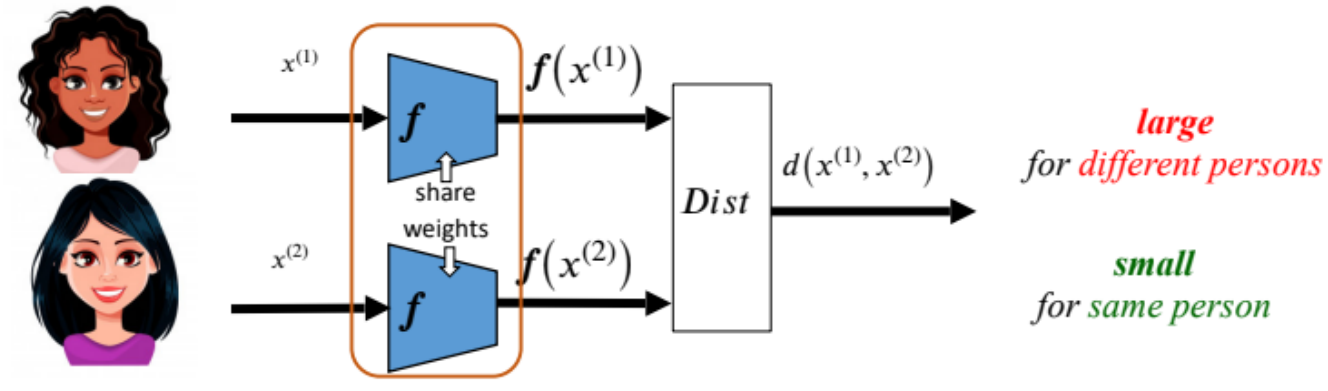


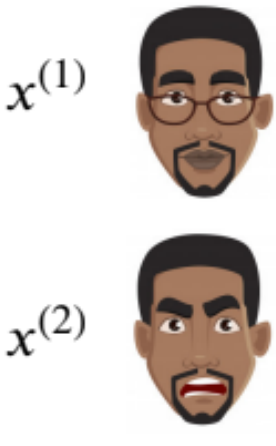
Figura 23: Exemplo de Rede Siamês

Cada imagem vai gerar um gradiente diferente nos parâmetros da rede. Para que ambas fiquem idênticas. Nós calculamos ambos os gradientes separadamente e depois fazemos uma média deles para atualizar os pesos da rede. Dessa forma, ambas as redes ficam idênticas.

2.4.1 Constrastive Loss

A loss utilizada para treinar redes siamesas é a **Contrastive Loss**. Vamos construí-la aos poucos.

Se A e B forem a mesma pessoa, gostaríamos que a loss gerasse um valor baixo, ou seja, que a distância entre os embeddings fosse pequena.



$$\mathscr{L}\left(x^{(1)}, x^{(2)}\right) = \left\| f\left(x^{(1)}\right) - f\left(x^{(2)}\right) \right\|^2$$

Figura 24: Exemplo de Contrastive Loss para imagens da mesma classe

Se A e B forem pessoas diferentes, gostaríamos que a loss gerasse um valor alto, ou seja, que a distância entre os embeddings fosse grande. No entanto, não é interessante que a loss seja muito grande, pois isso faria com que o modelo tentasse aprender a identificar características muito distintas. Intuitivamente, é como se ambas as imagens fossem tão diferentes, que não faz sentido o modelo tentar aprender todas as diferenças, não contribuem para o aprendizado. Então

definimos um **margin** m que determina o quão distante os embeddings devem estar. Ou seja, se a distância entre os embeddings for maior que o margin, a loss não precisa aumentar mais.

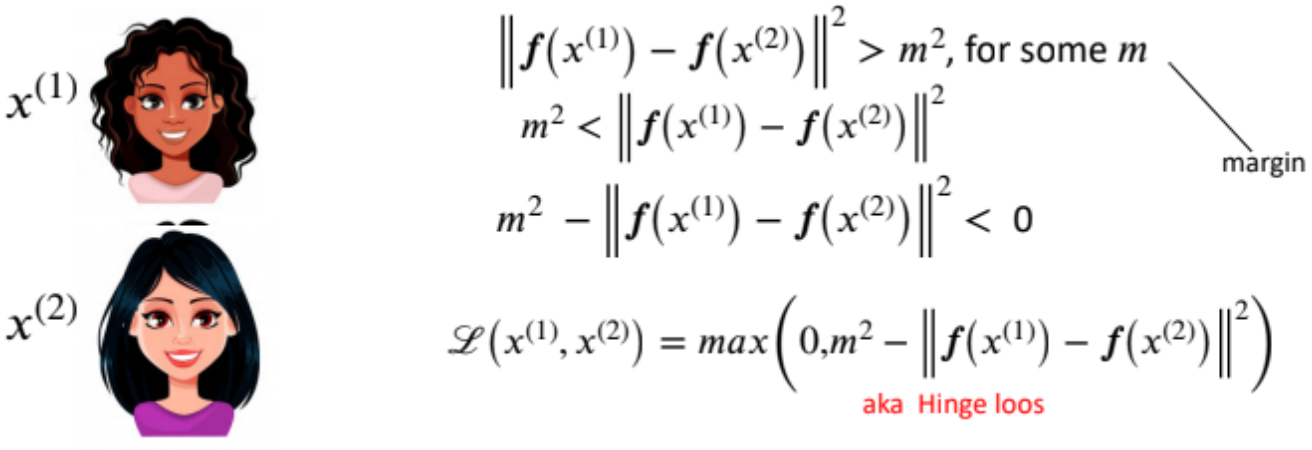


Figura 25: Exemplo de Contrastive Loss para imagens de classes diferentes

Então dado que t é uma variável indicadora que vale 1 se as imagens forem da mesma classe e 0 caso contrário, a loss final é definida como:

$$\mathcal{L} = t \cdot d^2 + (1 - t) \cdot \max(0, m^2 - d)^2 \tag{7}$$

onde d é a distância entre os embeddings e m é o margin. No problema de reconhecimento facial, d costuma ser a distância euclidiana.

2.4.2 Weighted Double Margin Contrastive (WDMC) Loss

Muito parecida com a contrastive loss. No entanto, adicionamos a lógica anterior de pares diferentes em pares iguais também. A lógica é que, se as duas imagens forem **muito** parecidas, não faz sentido fazer com que o modelo perca tempo aprendendo diferenças mínimas. Então definimos duas margens, m e M, uma para pares diferentes e outra para pares iguais.

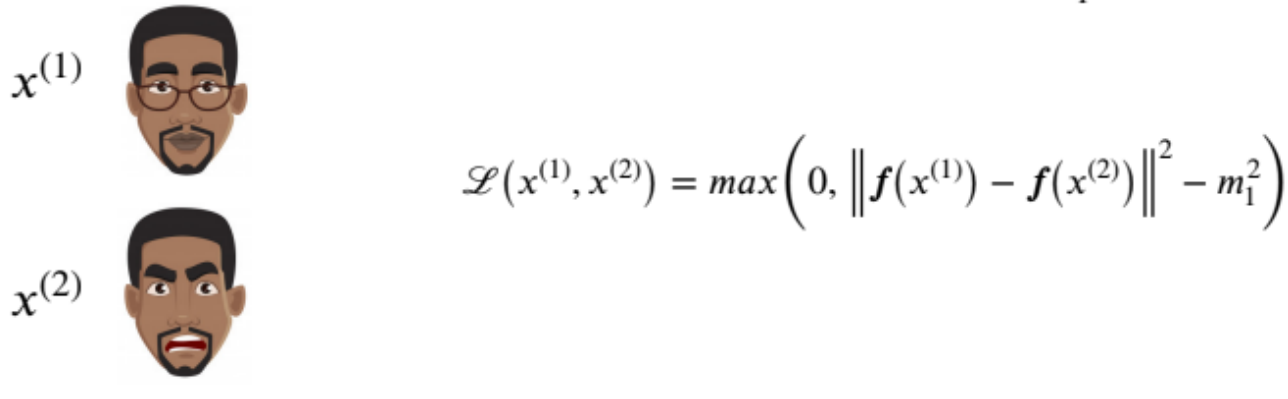


Figura 26: Exemplo de WDMC Loss

Dessa forma, a loss final é definida, para um batch $\left\{x_i^{(1)}, x_i^{(2)}\right\}_{i=1}^{N}$, como:

$$\mathcal{L} = \sum_i w_1 t_i \max\left(0, d\left(x_i^{(1)}, x_i^{(2)}\right) - m^2\right) + w_2(1 - t_i) \max\left(0, M^2 - d\left(x_i^{(1)}, x_i^{(2)}\right)\right) \tag{8}$$

2.4.3 Loss Tripla

Em vez de treinarmos em pares de imagens, podemos treinar em **trios** de imagens. Temos uma imagem de referência (Anchor A), uma imagem da mesma classe (Positive P) e uma imagem de

classe diferente (Negative N). A ideia é que a distância entre o Anchor e o Positive seja menor que a distância entre o Anchor e o Negative. Ou seja, queremos que:

$$d(A, P) < d(A, N) \tag{9}$$

logo, deixando mais restrito para uma margem m, queremos

$$d(A, P) - d(A, N) < -m \tag{10}$$

então temos que

$$\mathcal{L} = \max(0, d(A, P) - d(A, N) + m) \tag{11}$$

Para um batch $\{A_i, P_i, N_i\}_{i=1}^{N}$, a loss final é definida como:

$$\mathcal{L} = \sum_{i} \max(0, d(A_i, P_i) - d(A_i, N_i) + m) \tag{12}$$

A arquitetura vira um conjunto de 3 redes siamesas, uma para cada imagem, que compartilham os mesmos pesos e parâmetros. A loss é calculada a partir das distâncias entre os embeddings gerados pelas redes.

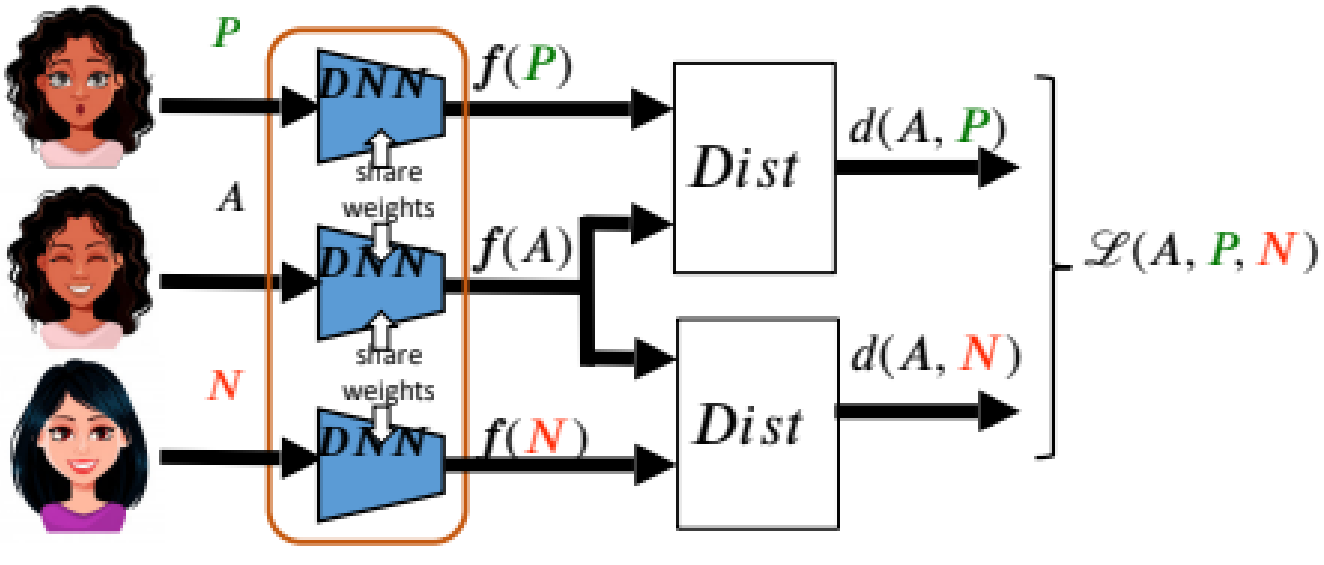


Figura 27: Exemplo de arquitetura para a Triplet Loss

E para a fase de **testes**, utilizamos a estrutura com apenas duas redes siamesas, uma para a imagem de referência e outra para a imagem a ser comparada.

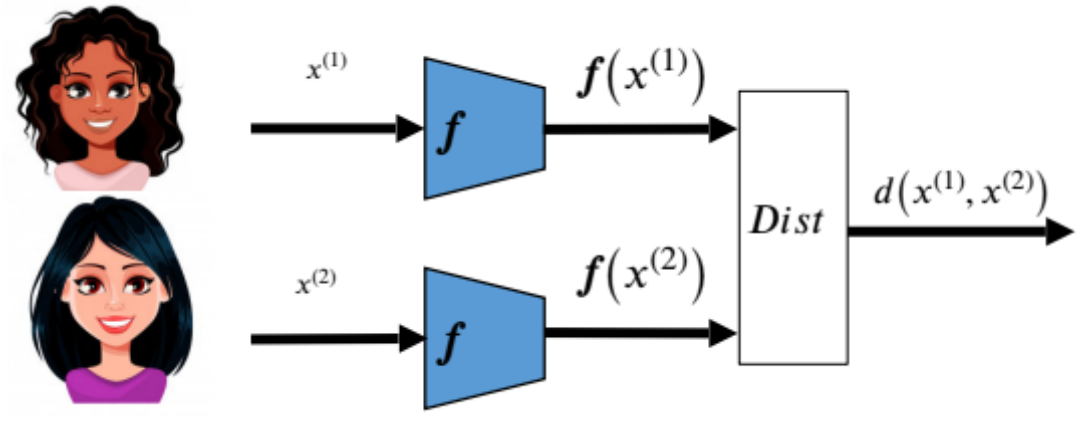


Figura 28: Exemplo de arquitetura para a Triplet Loss na fase de teste

2.5 Improving Similarity Learning

Existem alguma forma de melhorar o aprendizado por similaridade. Alguns pontos que vamos abordar são:

1. **Melhorar a Loss**
2. **Melhorar a Amostragem**
3. **Ensenble Methods**

A parte de melhoria da loss nós já comentamos com as abordagens de diferentes loss anteriormente (mesmo que ainda existem falhas que vão ser apontadas)! Então vamos focar nos outros pontos.

2.5.1 Melhorar a Amostragem

Vamos nesse momento, dar um pouco de foco na forma de abordagem usando **triplas** de imagens. O problema de trabalhar com triplas, é que o número total de combinações possíveis em um conjunto com n dados é da ordem de n^3. Ou seja, tanto você sintetizar todos esses trios, quanto você **treinar** com eles vai demorar bastante. Ainda existe o fato que, utilizando da triplet loss, **nem todos eles são úteis para o aprendizado**. Lembrando que para uma tripla específica:

$$\mathcal{L}(A, P, N) = \max(0, d(A, P) - d(A, N) + m) \tag{13}$$

então o gradiente dessa função fica

$$\frac{\mathrm{d}\max}{\mathrm{d}x} = \begin{cases} 0 & x \leq 0 \\ 1 & x > 0 \end{cases} \tag{14}$$

ou seja, para triplas onde a seguinte condição é verdade:

$$d(A, P) - d(A, N) + m \leq 0 \tag{15}$$

não vão contribuir para o aprendizado, pois o gradiente é zero!

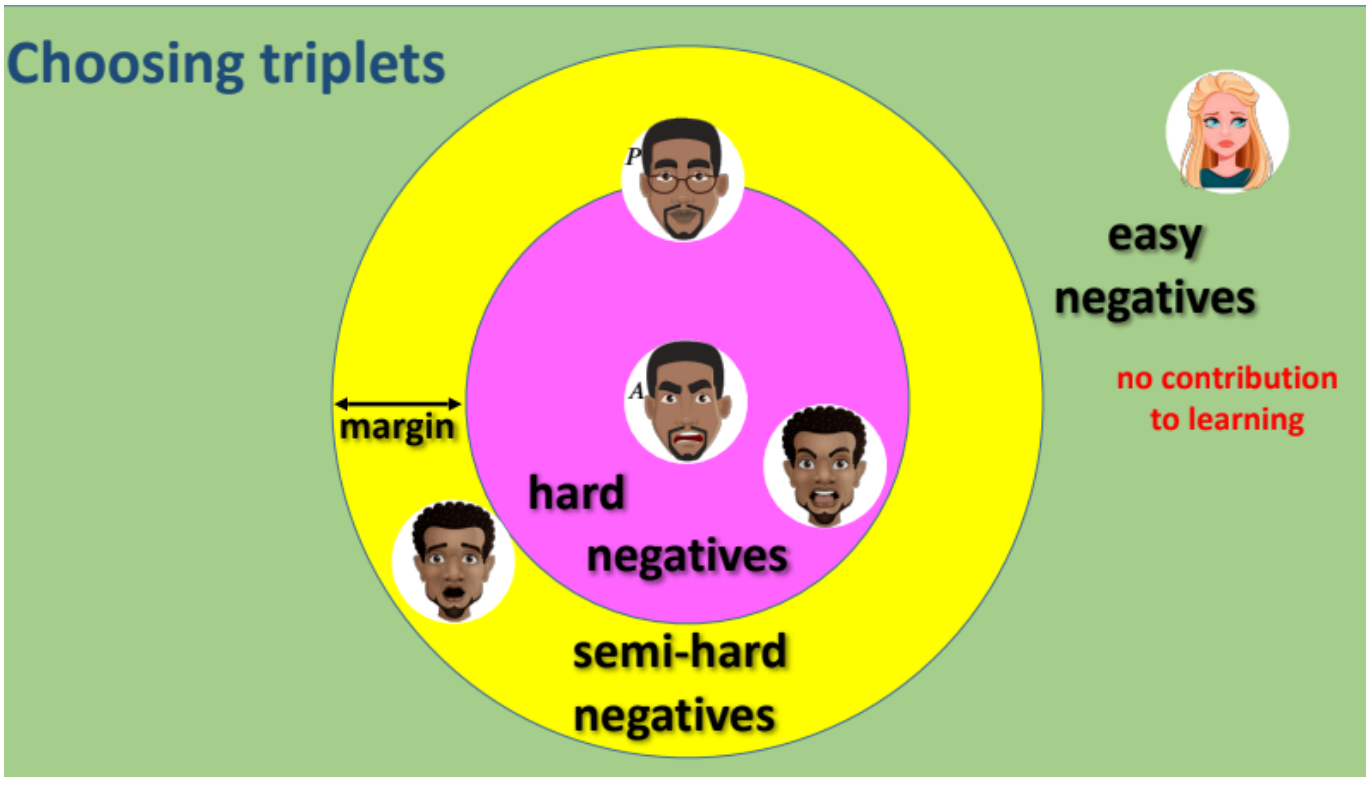


Figura 29: Exemplo de gradiente da Triplet Loss

Pela ilustração da imagem, queremos pegar exemplos **difíceis** ou **semi-difíceis**, pois eles geram os gradientes mais fortes para as losses. Um exemplo ingênuo de seleção de triplas, seria algo como:

selecionar triplas iniciais $\{A_i, P_i, N_i\}$
enquanto $\text{condição de parada} = \text{false}$ {
    Treina por algumas épocas
    Escolhe triplas difíceis $\{A_j, P_j, N_j\}$
}

As triplas podem ser selecionadas de forma offline (manualmente) ou online (automaticamente). Vamos tentar pensar em algum método pra seleção de triplas online.

A ideia é termos uma **árvore** de classes. Essa árvore vai servir como um **GPS** do dataset, agrupando as imagens por similaridade visual. Em vez de testar tudo contra tudo, a rede vai consultar essa árvore pra responder duas perguntas:

1. *Como garanto que o batch tenha classes variadas e não fique viciado em poucas classes repetidas?*
2. *Para cada imagem de* ***âncora****, onde estão os negativos que são difíceis de verdade, mas sem pegar ruído?*

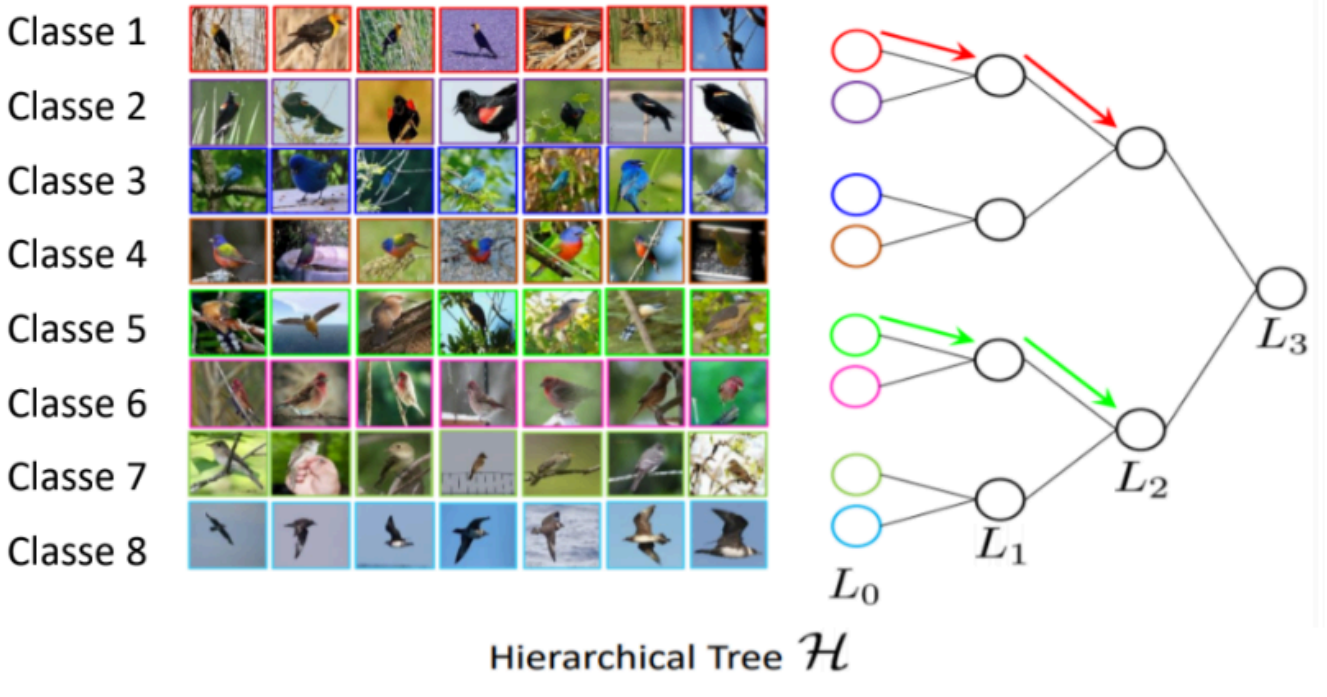


Figura 30: Exemplo de árvore de classes para seleção de triplas

Vamos definir o processo de escolha na árvore antes de estabelecermos como ela deve ser construída. O algoritmo consiste em 3 passos. O básico de conhecimento prévio que necessitamos é que, para cada **folha** da árvore, elas representam uma **classe**; e para cada **nó** da árvore, ele representa um **grupo de classes**.

```
function select_triplets(tree, batch_size) {
    seleciona l' classes aleatórias da árvore (folhas)
    for each âncora A {
        pego o positivo mais longe do âncora A (dentro da mesma classe)
        utilizo a árvore para achar as k classes vizinhas mais próximas da classe do âncora A e sorteio t negativos
    }
    return batch montado B ∈ ℝ^{l' × k × t}
}
```

Algoritmo 31: Exemplo de algoritmo para seleção de triplas na árvore

Agora que entendemos como e o porquê utilizamos a árvore, podemos compreender sua construção.

O primeiro passo para a construção da árvore, é a definição dos nó folha, onde cada um consiste em uma classe diferente do dataset. Depois, temos que definir a **between class distance**, que é a distância entre duas classes. Podemos utilizar, por exemplo:

$$d(p,q) = \frac{1}{n_p n_q} \sum_{i \in p} \sum_{j \in q} \|r_i - r_j\|^2 \tag{16}$$

onde r_k é o embedding da imagem k gerada pela rede neural.

Perfeito, agora precisamos definir uma medida de distância entre as imagens dentro da **mesma classe**, que chamaremos de **within class distance**. Podemos utilizar, por exemplo:

$$d_0 = \frac{1}{C} \sum_{c=1}^{C} \left(\frac{1}{n_c^2 - n_c} \sum_{i,j \in c} \|r_i - r_j\|^2 \right) \tag{17}$$

onde C é o número de classes e n_c é o número de imagens da classe c.

Agora, vamos fazer, recursivamente do nível $l = 0$ até $l = L$, uma **fusão** dos nós, de forma que os nós que satisfazerem um critério específico, vão ser unidos. Mas qual critério utilizamos?

$$d_l = \frac{l(4 - d_0)}{L} + d_0 \tag{18}$$

utilizamos dessa distância e, se duas classes possuem suas distâncias menores que d_l, elas são unidas. Esse processo se repete até o nível $l = L$.

2.5.2 Melhorar a Loss

Relembrando a nossa loss tripla, temos que:

$$\mathcal{L} = \sum_i \max(0, d(A_i, P_i) - d(A_i, N_i) + m) \tag{19}$$

O problema que temos nessa loss é que, por conta que a margem é fixa, muitas vezes, o modelo ignora triplas que podem ser de alguma forma úteis, seja por que a diferença da margem seja mínima, seja para exemplos mais difíceis mas que poderiam ajudar o modelo a compreender diferentes graus de parentesco entre as imagens ou variabilidade interna

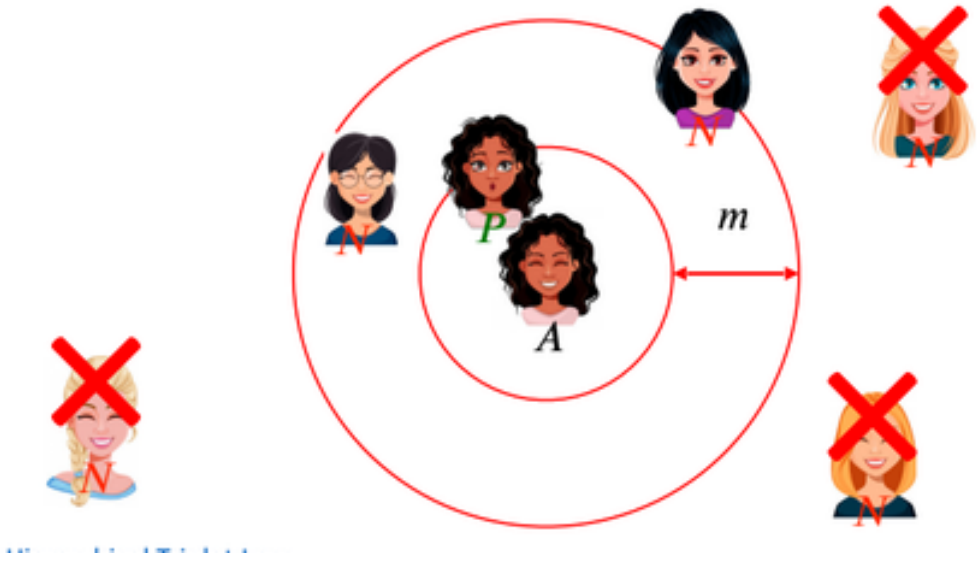


Figura 32: Exemplo da margem excluindo triplas úteis

A ideia é modificar a loss para que a margem fique dependente da distância entre a âncora e os exemplos negativos calculados na árvore hierárquica. Dessa forma, a loss final fica:

$$\mathcal{L} = \sum_i \max(0, d(A_i, P_i) - d(A_i, N_i) + m_i) \tag{20}$$

As distâncias mudam conforme o treino progride, logo, a árvore deve ser recalibrada a cada k épocas.

2.5.3 Ensemble Methods (Divide and Conquer)

Estamos trabalhando com um problema de redes neurais gerarem funções de um espaço $\mathbb{R}^n$ para um latente $\mathbb{R}^d$, de forma que, no espaço latente, os objetos parecidos fiquem próximos e os diferentes fiquem distantes. No entanto, a noção de parecido e diferente é **multifacetada**. Considere o caso dos rostos humanos, por exemplo, temos **cor de pele**, **formato do rosto**, **cor dos olhos**, **formato do nariz**, **formato da boca**, etc. Fazer com que **um único modelo** aprenda todas essas características pode sobrecarregar ele e atrapalhar seu aprendizado. A ideia do **dividir e conquistar** é que, em vez de uma única rede, combinamos múltiplos **learners** que juntam suas soluções em um espaço latente unificado.

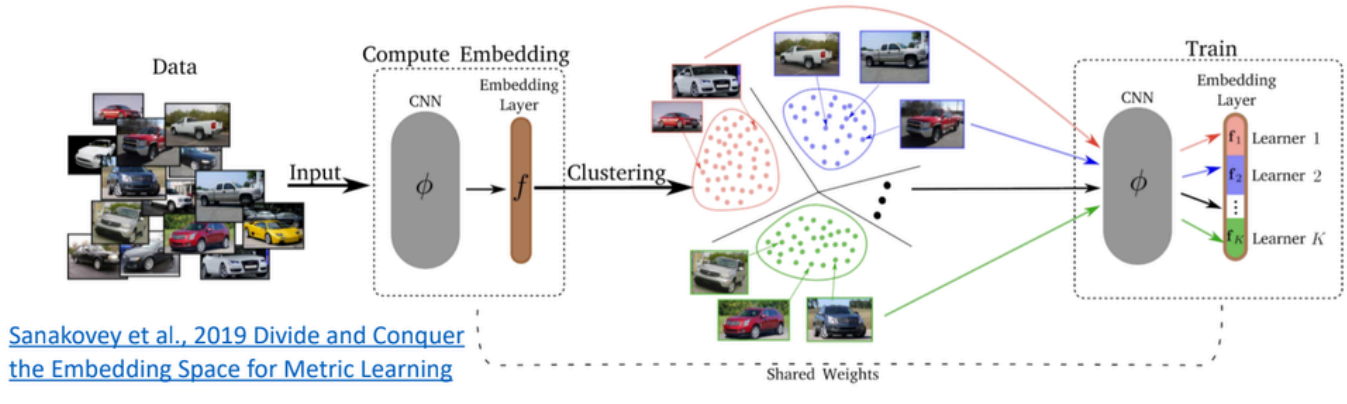


Figura 33: Exemplo de Multiple Learners

A primeira coisa que é feita é passar a imagem de entrada em uma CNN backbone **comum à todos os learners** (mesmos parâmetros em todos) que extrai as features brutas da imagem para que cada learner possa processar separadamente. Dos meus K learners, cada um tem sua própria camada linear que projeta a saída dessa backbone em um subespaço latente de dimensão d/K.

O algoritmo é dividido em duas fases, a **divisão** e a **conquista**. Na fase de divisão, temos 4 passos principais:

1. **Agrupamento (K-means)**: tilizando o espaço de embedding atual, as amostras do dataset são divididas em K clusters por meio do algoritmo K-Means. Cada cluster agrupa dados que compartilham noções de similaridade mais homogêneas (por exemplo, um cluster para picapes, outro para sedans, outro para SUVs)
2. **Definição dos K Learners**: Criam-se K learners, onde cada learner k é composto pela CNN compartilhada φ seguida pela sua projeção f_k para dimensão $\frac{d}{K}$
3. **Treinamento Alternado**: Os aprendizes são treinados alternadamente:
   - O aprendiz k é alimentado apenas com amostras do seu respectivo k-ésimo cluster.
   - Em cada retropropagação (backward pass), o gradiente atualiza os pesos da CNN compartilhada φ e os pesos da camada linear local f_k.
4. **Reagrupamento Periódico**: Como a representação de φ evolui com o treinamento, a cada T épocas todos os dados são re-clusterizados no espaço de embedding global atualizado

Já na fase de **conquista**, temos 2 passos principais:

1. **Fusão dos Aprendizes**: Após a convergência dos K aprendizes, as representações parciais (de dimensão $\frac{d}{K}$) são unidas/concatenadas para recompor o espaço de embedding completo de dimensão d
2. **Ajuste Fino Global (Fine-Tuning)**: Realiza-se um **fine-tuning** da camada de embedding em todo o conjunto de dados para garantir a **consistência mútua** entre as representações geradas individualmente por cada aprendiz

Aprendendo em Dados Não-Rotulados